Radar de Oportunidades

Cómo funciona el módulo — resumen ejecutivo (implementación real)

MGA Informática · mgadigital.com.ar · Doc generada: oct 2026 · Fuentes:
`context/modulos/radar_oportunidades_spec.md` , `app/(dashboard)/dashboard/radar/` , `lib/radar/` , `app/api/dashboard/radar/` , `supabase/migrations/radar_oportunidades.sql`

1. Objetivo del módulo

Herramienta comercial interna que busca negocios y empresas potenciales por **ubicación y rubro** usando **Google Places**, analiza su presencia digital y les asigna un **puntaje de oportunidad** para priorizar el contacto y ofrecer servicios de desarrollo, automatizaciones y sistemas.

- Busca comercios (prospectos) por localidad + rubro + palabras clave.
- Los deduplica, los puntúa y los ordena por oportunidad.
- Permite seguimiento comercial (estados, interacciones) y **convertir un prospecto en cliente** real.
- No contacta automáticamente: el usuario revisa y actúa (manual, por diseño — revisión humana).

2. Flujo real de una búsqueda

> **Importante:** el proceso es **100 % síncrono dentro del POST** — no hay jobs, colas ni workers activos (la tabla `prospect_jobs` existe pero nunca se usa). El formulario queda en "Buscando..." hasta que termina todo.

1. **Formulario** — `radar/nueva-busqueda/NuevaBusquedaClient.tsx` : provincia, localidad, rubro, keywords, radio, maxResults, servicio a ofrecer, filtros (teléfono, email, web, rating, reseñas).
2. `POST /api/dashboard/radar/searches` — crea la fila en `prospect_searches` con estado `searching` .
3. **Genera múltiples consultas** — `buildQueries()` : 1 por rubro ("*centros de estética en Comodoro Rivadavia*") + 1 por cada keyword + fallback "*lugares en {localidad}*".
4. **Google Places Text Search** — hasta 20 por consulta, paginación con `pagetoken` (espera 1.5 s por página, máx. 10 páginas). Key sólo en servidor (`GOOGLE_MAPS_API_KEY`).
5. **Por cada resultado (`processResult`):**
    1. Filtro de categoría (`categoryFilter.ts` — descarta false positives: bancos, escuelas, comida, etc.)
    2. Extrae email del sitio web *(sólo si se pidió email en los filtros)*
    3. Filtro de contacto (teléfono/email requeridos)
    4. **Deduplicación** contra prospectos existentes
    5. **Cálculo de los 4 puntajes + puntaje final** y motivos
    6. Inserta en `prospects` (estado `ready`) + `prospect_search_results`
6. **Progreso en BD** — actualiza `total_found / total_saved / progress` (pero nadie lo lee en vivo: sin Realtime ni polling).

7. **Finaliza** — `completed` (hubo resultados) o `partial` (0 resultados) → el cliente navega a `/dashboard/radar?searchId=…` y carga la lista ordenada por `opportunity_score` desc.

Deduplicación (prioridad exacta)

1. `google_place_id`
2. Teléfono normalizado (sólo dígitos)
3. Dominio del sitio web
4. Nombre normalizado + localidad
5. Dirección normalizada

Además hay índice único parcial `prospects(user_id, google_place_id)` como red de seguridad.

3. Motor de puntajes

Cada prospecto recibe 4 sub-puntajes (0–100) y un puntaje final ponderado:

30%
Actividad comercial
Reseñas (≥100: +30), rating (≥4.5: +15), operativo +15, teléfono/web/horarios +10 c/u

35%
Necesidad digital
Sin web → 90 directo. Con web: inaccesible +30, sin SSL +15, no mobile +20, perf baja, sin catálogo/turnos/formulario

15%
Contactabilidad
WhatsApp +30, teléfono +25, email +25, formulario +10, dirección +10

20%
Compatibilidad
Match rubro ↔ servicios configurados; sin servicios cargados → 50 fijo

```
opportunity_score = round(actividad*0.30 + necesidad*0.35 + contacto*0.15 + compat*0.20)
```

Nivel	Rango	Traducción UI
`very_high`	≥ 80	Muy alta
`high`	≥ 65	Alta
`medium`	≥ 45	Media
`low`	≥ 25	Baja
`very_low`	< 25	Muy baja

Motivos explicables: cada regla genera un registro `{code, label, impact, category}` que se guarda en `opportunity_reasons` (JSONB) y se muestra en el detalle con su impacto `+N`. Nunca se muestra un número sin explicación.

> Los pesos viven en la constante `DEFAULT_SCORING_WEIGHTS` (`types/radar.ts`) — no en configuración de BD.

4. Página de Prospectos

`radar/prospectos/page.tsx` (SSR con auth + permisos del módulo `radar`) + `ProspectosClient.tsx` :

Aspecto	Detalle
Filtros	Texto libre (nombre/rubro/ciudad/tel/email), **estado comercial** (12 estados), ciudad, rubro (combobox con catálogo), rating mínimo (4+/3+/2+)
Ordenamiento	Clic en cabecera: Nombre, Rubro, Localidad, WhatsApp, Teléfono, Email, Estado, Rating, Score, Creado
Paginación	15 por página (Anterior/Siguiente)

Columnas	Nombre, Rubro, Localidad, WhatsApp, Teléfono, Email, Estado (badge), Rating, Score, Creado, Acciones
Acciones por fila	Ver detalle · **WhatsApp** con mensaje precargado de MGA · **Email** (mailto:) · Abrir sitio web
API	`GET /api/dashboard/radar/prospects` con `page, pageSize, sortBy, sortDirection, search, commercialStatus, city, rubro, minRating`

Vista de detalle (`prospectos/[id]`)

1. **Header** — nombre, selector de **estado comercial** (PATCH al cambiar), botón **"Convertir a cliente"** o badge "Cliente propio", botón WhatsApp con mensaje precargado.
2. **Información general** — rubro, ciudad/provincia, dirección, estado del negocio, rating + reseñas.
3. **Contacto** — teléfono (tel:), **email y WhatsApp editables inline**, sitio web, link a Google Maps.
4. **Puntajes** — círculo con score final + nivel + servicio recomendado, los 4 bloques de sub-puntajes y los **motivos con impacto**.
5. **Historial de seguimiento** — interacciones (teléfono/whatsapp/mail/presencial/otro) con fecha, canal, notas y próximo seguimiento; alta y baja desde la misma pantalla.
6. **Análisis del sitio web** — sección condicional (casi nunca se llena: ver brechas).
7. **Fuente** — origen (google_places), última verificación, estado comercial.

5. API y base de datos

Rutas (todas con `auth()` + `supabaseAdmin`, filtrando `user_id`)

Ruta	Métodos	Función
`/api/dashboard/radar/searches`	GET · POST	Listar / crear búsqueda (ejecuta todo el pipeline)
`/api/dashboard/radar/searches/[id]`	GET · DELETE	Detalle / eliminar búsqueda
`/api/dashboard/radar/prospects`	GET	Lista con filtros, orden y paginación
`/api/dashboard/radar/prospects/[id]`	GET	Detalle del prospecto
`…/prospects/[id]/status`	PATCH	Estado comercial, email y WhatsApp
`…/prospects/[id]/interactions`	POST	Registrar interacción
`…/prospects/[id]/convert`	POST	Convertir prospecto en cliente/servicio/presupuesto
`/api/dashboard/radar/servicios`	GET · POST · PUT · DELETE	CRUD de servicios del radar (con categorías destino)
`/api/dashboard/radar/interactions/[id]`	DELETE	Borrar interacción
`/api/dashboard/radar/usage`	GET	Consumo/costos de la API de Google

Tablas Supabase (`supabase/migrations/radar_oportunidades.sql`)

Tabla	Uso
`prospects`	Núcleo: datos del negocio, contacto, 4 puntajes + score final, motivos (JSONB), estados, fuente
`prospect_searches`	Criterios, estado, contadores y progreso de cada búsqueda
`prospect_search_results`	Nexo búsqueda ↔ prospecto (con query que lo matcheó)
`prospect_services`	Servicios ofrecidos y sus categorías/palabras destino
`prospect_interactions`	Historial de seguimiento comercial
`website_analyses`	Resultado del análisis web (sólo lectura hoy)
`radar_api_usage`	Registro de llamadas/costos a Google
`rubros`	Catálogo de rubros (24 sembrados)
`prospect_jobs`	Definida pero **sin uso**

Todas con **RLS** habilitado (políticas por `auth.uid() = user_id`); el dashboard usa el cliente service-role en servidor.

Estados

- **Búsqueda:** `searching → completed | partial | failed` (los estados `pending/enriching/scoring/cancelled` están declarados pero no se usan).
- **Prospecto (análisis):** se crea directo en `ready` (sin fase `pending_analysis` real).
- **Comercial:** `new → reviewed → to_contact → contacted → interested → meeting → proposal_sent → won/lost/discarded → do_not_contact`, más `existing_customer` (lo setea "Convertir a cliente").

6. Brechas: spec vs realidad

Spec propone	Hoy	Estado
Procesamiento asíncrono con jobs/colas/worker	Todo síncrono en el POST; `prospect_jobs` vacía	No
Progreso en tiempo real (Supabase Realtime)	Sin Realtime ni polling; `progress` se escribe pero no se lee	No
Exportación a CSV	No existe	No
Reanalizar prospecto	No existe endpoint ni botón	No
PageSpeed Insights + CrUX	Sin llamadas (key "future" en .env.example); el +10 de CrUX nunca aplica	No
Enriquecimiento web: teléfonos, WhatsApp, formularios, catálogo, e-commerce, turnos, tecnologías	Sólo extrae **emails** (`website.ts` , 42 líneas)	Parcial
Guardar con `pending_analysis` y analizar después	Se guarda directo en `ready`	Parcial
Filtros de formulario (web, rating, reseñas, estado) aplicados a la búsqueda	Se persisten pero **no filtran** los resultados; sólo filtran rubro, keywords, teléfono, email	Parcial
Radio geográfico	`radiusKm` se envía sin `location` → efectivamente ignorado	Parcial
Pesos de scoring configurables en BD	Constante TS	Parcial
Protección SSRF en el analizador web	No se valida host/IP privada; sólo timeout 8 s	No
Scoring completo con todos los inputs	`service_fit` siempre 50 · `digital_need` sólo 90 o 20 · WhatsApp nunca puntúa	Parcial
Exclusión / no contactar (UI + API)	Columnas existen en BD, sin UI ni API	No

Lo que sí cumple la spec

- Deduplicación con la prioridad exacta y multi-query por rubro + keywords.
- Pesos 0.30/0.35/0.15/0.20, 5 niveles de oportunidad y motivos guardados/explicados.
- Errores individuales no frenan la búsqueda; key de Google sólo en servidor.
- Filtros + ordenamiento + paginación en la lista; RLS en todas las tablas.
- Estados comerciales de la spec + extras: conversión a cliente, tracking de costos y catálogo de rubros.

> **En síntesis:** el Radar funciona como un buscador con scoring explicable que corre de punta a punta en un solo request. El pipeline de análisis profundo (web, PageSpeed, jobs, realtime, CSV) está sólo esbozado en tablas y tipos — es la zona natural de evolución del módulo.

Documento resumen generado para referencia rápida. La especificación completa (33 secciones) está en `context/modulos/radar_oportunidades_spec.md`.